

The following categories of data sets are available in TFDS: audio, image, image classification, object detection, questions and answers, structured, summarisation, text, translate, video and vision language. Each of these data sets can be used for a variety of machine learning and deep learning tasks.

Installation of these data sets can be done from two packages available with TFDS.

pip install tensorflow-datasets: This is the stable version, released every few months.

pip install tfds-nightly: Released every day, it contains the latest versions of the data sets.

We can import the data sets using the following commands:

```
import tensorflow as tf
import tensorflow_datasets as tfds
```

All data set builders are a subclass of *tfds.core.DatasetBuilder*. To get the list of available builders, use *tfds.list_builders()*.

The following example taken from *tensorflow.org* explains the usage of the TensorFlow data set MNIST.

1) Load MNIST

Load with the following arguments.

- **shuffle_files:** The MNIST data is only stored in a single file, but for larger data sets with multiple files on disk, it's a good practice to shuffle them when training.
- **as_supervised:** Returns a tuple (*img*, *label*) instead of dict {'image': *img*, 'label': *label*} (*ds_train*, *ds_test*), *ds_info* = *tfds.load*('mnist', *split*=['train', 'test'], *shuffle_files*=True, *as_supervised*=True, *with_info*=True,)

2) Build training pipeline

Apply the following transformations.

- **ds.map:** TFDS provides the images as *tf.uint8*, while the model expects *tf.float32*; so normalise the images.
- **ds.cache:** As the data set fits in

memory, cache before shuffling for better performance.

- **ds.shuffle:** For true randomness, set the shuffle buffer to the full data set size.



Note: For bigger data sets that do not fit in memory, a standard value is 1000 if your system allows it.

- **ds.batch:** Batch after shuffling to get unique batches at each epoch.
- **ds.prefetch:** It's a good practice to end the pipeline by prefetching for performances.

```
def normalize_img(image, label):
    """Normalizes images: `uint8` ->
    `float32`."""
    return tf.cast(image, tf.float32) /
    255., label
ds_train = ds_train.map( normalize_
img, num_parallel_calls=tf.data.
experimental.AUTOTUNE) ds_
train = ds_train.cache()
ds_train = ds_train.shuffle(ds_info.
splits['train'].num_examples)
ds_train = ds_train.batch(128)
ds_train = ds_train.prefetch(tf.data.
experimental.AUTOTUNE)
```

3) Build evaluation pipeline

Testing pipeline is similar to the training pipeline, with a small difference — there is no *ds.shuffle()* call.

Caching is done after batching (as batches can be the same between epochs):

```
ds_test = ds_test.map( normalize_
img, num_parallel_calls=tf.data.
experimental.AUTOTUNE)
ds_test = ds_test.batch(128)
ds_test = ds_test.cache()
ds_test = ds_test.prefetch(tf.data.
experimental.AUTOTUNE)
```

4) Create and train the model

Plug the input pipeline into Keras:

```
model = tf.keras.models.
Sequential([tf.keras.layers.
```

```
Flatten (input_shape=(28, 28)),
tf.keras.layers.Dense
(128,activation='relu'), tf.keras.
layers.Dense(10)]), model.compile(
optimizer=tf.keras.optimizers.
Adam(0.001), loss=tf.keras.losses.
SparseCategoricalCrossentropy(fr
om_logits=True), metrics=[tf.keras.
metrics.SparseCategoricalAccuracy()],)
model.fit( ds_train, epochs=5,
validation_data=ds_test,)
```

Using the data sets of PyTorch

Like any other ML or DL framework, PyTorch also has built-in data sets that can be explored for various applications. To perform ETL (extract, transform and load) on a given data set, PyTorch provides the two main classes given below.

- **Dataset:** This is an abstract class representing a data set.
- **DataLoader:** This is a Python iterable over the data set; so it wraps a data set and provides access to underlying data.

The Dataset abstract class has two methods, *__len__()* and *__getitem__()*, which need to be implemented for custom data sets by extending this class. The data sets can be passed to DataLoader object to load multiple samples in parallel by using multiprocessing worker modules of PyTorch.

The details of how to create a new data set and use it in PyTorch are given below, using the classes discussed above.

To generate a new data set, for example “*newDataset*”, it needs to be extended from the built-in abstract class *torch.utils.data.Dataset*. Post that, the initialisation *__init__()*, *__len__()* and *__getitem__()* functions need to be overridden and the corresponding implementation needs to be provided. A sample code to illustrate the defining of the *newDataset* class using the Dataset abstract class of PyTorch is given below:

```
import torch
class newDataset(torch.utils.data.
Dataset):
    # Initialization Function
    def __init__(self, listIDs, labels):
```